

Algoritmi Clasici de Criptare - Referat

1. Introducere și Obiective

Referatul prezintă rezolvarea cerințelor practice stabilite în cadrul temei '*Referat1_Criptografie-cerințe*'. Obiectivul principal constă în proiectarea și implementarea de la zero a unor algoritmi de criptografie clasici (fără utilizarea unor librării criptografice dedicate).

Aplicația este dezvoltată nativ în **C++** și respectă o arhitectură modulară curată: fiecare algoritm are dedicat o interfață proprie (fișierele `.h`) și o secțiune izolată de implementare a logicii matematice (fișierele `.cpp`) toate fiind organizate dinamic printr-un meniu interactiv controlat din terminal (`Menu.cpp`).

2. Structura Proiectului

```
src/  
|— main.cpp  
|— Menu.cpp  
|— Menu.h  
|— Caesar.cpp  
|— Caesar.h  
|— Vigenere.cpp  
|— Vigenere.h  
|— Substitution.cpp  
|— Substitution.h  
|— Transposition.cpp  
|— Transposition.h  
|— Mixed.cpp  
|— Mixed.h
```

La finalul documentului se află și linkul către repo-ul de pe GitHub.

3. Descrierea Algoritmilor și a Fundamentelor Matematice

3.1. Cifrul Caesar

Descriere

Una dintre cele mai vechi metode de criptare. Fiecare literă este deplasată în alfabet cu un număr fix de poziții reprezentat de o cheie.

Formule

- **Formula de Criptare:** $E(x) = (x + k) \pmod{26}$
- **Formula de Decriptare:** $D(y) = (y - k + 26) \pmod{26}$ (Adunarea valorii 26 previne rezultatele negative în operatorul modular din C++).

Fragment din cod

```
// Sursă: Caesar.cpp
std::string caesarEncrypt(std::string text, int key) {
    std::string result;
    for(char c : text) {
        if(isalpha(c)) {
            c = toupper(c);
            int x = c - 'A';
            int y = (x + key) % 26;
            result += char(y + 'A');
        } else { result += c; }
    }
    return result;
}
```

Exemplu

Mesaj:

CRIPTOGRAFIE

Cheie:

3

Rezultat:

Criptat: FULSWRJUDILH

Decriptat: CRIPTOGRAFIE

Capturi de ecran

```
std::string caesarEncrypt(std::string text, int key) {
    std::string result;

    for(char c : text)
    {
        if(isalpha(c)) // procesează doar caracterele alfabetice
        {
            c = toupper(c); // standardizare la majuscule

            int x = c - 'A'; // conversia caracter -> index
            int y = (x + key) % 26; // aplicarea formulei de criptare

            result += char(y + 'A'); // conversie index -> caracter
        }
        else
        {
            result += c; // păstrarea intactă a caracterelor speciale și a spațiilor
        }
    }

    return result;
}
```

```

std::string caesarDecrypt(std::string text, int key) {
    std::string result;

    for(char c : text)
    {
        if(isalpha(c))
        {
            c = toupper(c);

            int y = c - 'A';
            int x = (y - key + 26) % 26; // reface indexul original

            result += char(x + 'A');
        }
        else
        {
            result += c;
        }
    }

    return result;
}

```

```

(base) Madalinas-MacBook-Air:output madalinateaca$ ./Referat1
2. Vigenere
3. Substituție
4. Transpoziție
5. Sistem mixt
0. Ieșire

Opțiune: 1

Textul pentru criptare:
CRIPTOGRAFIE

Cheia pentru criptare (0-25):
3

Textul criptat:
FULSWRJUDILH
Textul decriptat:
CRIPTOGRAFIE

===== CRIPTO-R1 =====
1. Caesar
2. Vigenere
3. Substituție
4. Transpoziție
5. Sistem mixt
0. Ieșire

```

3.2. Cifrul Vigenère

Descriere

Cifrul Vigenère utilizează un cuvânt-cheie pentru a determina deplasarea aplicată fiecărei litere din mesaj. Acesta reprezintă o metodă de criptare polialfabetică.

Formule

- **Formula de Criptare:** $c_j = (m_j + k_{j \bmod n}) \bmod 26$
- **Formula de Decriptare:** $m_j = (c_j - k_{j \bmod n} + 26) \bmod 26$

Fragment din cod

```
// Sursă: Vigenere.cpp
std::string vigenereEncrypt(std::string text, std::string key) {
    std::string result;
    int keyIndex = 0;
    for(char c : text) {
        if(isalpha(c)) {
            c = toupper(c);
            int x = c - 'A';
            char k = toupper(key[keyIndex % key.length()]);
            int shift = k - 'A';
            int y = (x + shift) % 26;
            result += char(y + 'A');
            keyIndex++;
        } else {
            result += c;
        }
    }
    return result;
}
```

Exemplu

Mesaj:

CRIPTARE

Cheie:

TEACA

Rezultat:

Criptat: WVIRTTVE

Decriptat: CRTPTARE

Capturi de ecran

```
std::string vigenereEncrypt(std::string text, std::string key)
{
    std::string result;
    int keyIndex = 0; // contorizează poziția din cheie, avansând doar la caractere valide

    for(char c : text)
    {
        if(isalpha(c))
        {
            c = toupper(c);
            int x = c - 'A';

            // determinarea caracterului corespunzător din cheie și a deplasării
            char k = toupper(key[keyIndex % key.length()]);
            int shift = k - 'A';

            // criptarea
            int y = (x + shift) % 26;
            result += char(y + 'A');

            keyIndex++; // indexul cheii avansează doar dacă a fost criptată o literă
        }
        else
        {
            result += c;
        }
    }

    return result;
}
```

```

std::string vigenereDecrypt(std::string text, std::string key)
{
    std::string result;
    int keyIndex = 0;

    for(char c : text)
    {
        if(isalpha(c))
        {
            c = toupper(c);
            int y = c - 'A';

            char k = toupper(key[keyIndex % key.length()]);
            int shift = k - 'A';

            int x = (y - shift + 26) % 26;
            result += char(x + 'A');

            keyIndex++;
        }
        else
        {
            result += c;
        }
    }

    return result;
}

```

===== CRIPTO-R1 =====

1. Caesar
2. Vigenere
3. Substituție
4. Transpoziție
5. Sistem mixt
0. Ieșire

Opțiune: 2

Textul pentru criptare:
CRIPTARE

Cheia pentru criptare (text):
TEACA

Textul criptat:
VVIRTTVE
Textul decriptat: CRIPTARE

3.3. Algoritmul de Substituție

Descriere

Metoda de substituție înlocuiește fiecare literă cu o altă literă conform unui alfabet de substituție prestabilit.

Observații

În implementarea realizată a fost utilizat un alfabet de substituție construit pe baza unei chei prestabilite.

Alfabet permutat generator: TEABCDFGHIJKLMNOPQRSUVWXYZ

Fragment din cod

```
// Sursă: Substitution.cpp
std::string substitutionEncrypt(std::string text) {
    std::string result;
    for(char c : text) {
        if(isalpha(c)) {
            c = toupper(c);
            int pos = ALPHABET.find(c);
            result += SUBSTITUTE[pos];
        } else {
            result += c;
        }
    }
    return result;
}
```

Capturi de ecran

```
// alfabetul standard utilizat pentru maparea indexului (0-25)
const std::string ALPHABET = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";

// alfabetul de substituție definit prin permutarea generată de cuvântul cheie
const std::string SUBSTITUTE = "TEABCDGFIJKLMNOPQRSUVWXYZ";

/*
=== Descriere ===
- criptează un text prin folosirea substituției monoalfabetice simple
- se elimină relația de ordine nativă
- fiecare literă este înlocuită cu litera ce se află pe același index în alfabetul 'SUBSTITUTE'
*/
std::string substitutionEncrypt(std::string text)
{
    std::string result;

    for(char c : text)
    {
        if(isalpha(c))
        {
            c = toupper(c);
            int pos = ALPHABET.find(c); // găsește poziția nativă
            result += SUBSTITUTE[pos]; // înlocuiește cu litera de pe aceeași poziție din noul alfabet
        }
        else
        {
            result += c;
        }
    }

    return result;
}
```

```
std::string substitutionDecrypt(std::string text)
{
    std::string result;

    for(char c : text)
    {
        if(isalpha(c))
        {
            c = toupper(c);
            int pos = SUBSTITUTE.find(c); // identifică indexul în alfabetul criptat
            result += ALPHABET[pos]; // recuperează caracterul original
        }
        else
        {
            result += c;
        }
    }

    return result;
}
```

```
===== CRIPTO-R1 =====
```

1. Caesar
2. Vigenere
3. Substituție
4. Transpoziție
5. Sistem mixt
0. Ieșire

Opțiune: 3

Textul pentru criptare:
CRIPTOGRAFIE

Textul criptat:
AQHOSNFQTDHC

Textul decriptat:
CRIPTOGRAFIE

3.4. Algoritmul de Transpoziție

Descriere

Metoda de transpoziție modifică poziția caracterelor în mesaj fără a modifica valorile acestora.

Mesajul este scris într-un tabel și apoi citit pe coloane pentru obținerea textului criptat.

Observații

Preprocesare: Conform cursului, spațiile și caracterele non-alfabetice sunt ignorate sau eliminate pentru compactarea textului.

Fragment din cod

```
// Sursă: Transposition.cpp
std::string transpositionEncrypt(std::string text, int columns) {
    std::string cleanText;
    std::string result;
    for(char c : text) {
        if(isalpha(c)) cleanText += toupper(c);
    }
    int length = cleanText.length();
    for(int col = 0; col < columns; col++) {
        for(int i = col; i < length; i += columns) {
            result += cleanText[i];
        }
    }
    return result;
}
```

Capturi de ecran

```
std::string transpositionEncrypt(std::string text, int columns)
{
    std::string cleanText;
    std::string result;

    // curățarea și standardizarea - păstrarea doar a literelor și transformarea lor în majuscule
    for(char c : text)
    {
        if(isalpha(c))
            cleanText += toupper(c);
    }

    int length = cleanText.length();

    // citirea pe coloane a matricei
    for(int col = 0; col < columns; col++)
    {
        for(int i = col; i < length; i += columns)
        {
            result += cleanText[i];
        }
    }

    return result;
}
```

```
std::string transpositionDecrypt(std::string text, int columns)
{
    int length = text.length();
    // determinarea numărului de rânduri prin rotunjire superioară
    int rows = (length + columns - 1) / columns;

    std::string result(length, ' ');
    int index = 0;

    for(int col = 0; col < columns; col++)
    {
        for(int row = 0; row < rows; row++)
        {
            int position = row * columns + col;

            if(position < length)
            {
                result[position] = text[index];
                index++;
            }
        }
    }

    return result;
}
```

```
===== CRIPTO-R1 =====
1. Caesar
2. Vigenere
3. Substituție
4. Transpoziție
5. Sistem mixt
0. Ieșire

Opțiune: 4

Textul pentru criptare:
criptografie
Numărul de coloane (minim 1): 4

Textul criptat:
CTAROFIGIPRE

Textul decriptat:
CRIPTOGRAFIE
```

3.5. Sistemul Mixt (Produs de Cifruri)

Descriere

Sistemul mixt combină două metode de criptare:

1. ***Substituția monoalfabetică***
2. ***Transpoziția***

La decriptare se aplică operațiile în ordine inversă.

Fragment din cod

```
// Sursă: Mixed.cpp
std::string mixedEncrypt(std::string text, int columns) {
    std::string substituted = substitutionEncrypt(text);
    std::string encrypted = transpositionEncrypt(substituted, columns);
    return encrypted;
}
```

Capturi de ecran

```
std::string mixedEncrypt(std::string text, int columns)
{
    // aplică transformarea de substituție a caracterelor
    std::string substituted = substitutionEncrypt(text);
    // execută rearanjarea geometrică a pozițiilor
    std::string encrypted = transpositionEncrypt(substituted, columns);

    return encrypted;
}
```

```
std::string mixedDecrypt(std::string text, int columns)
{
    std::string transposedBack = transpositionDecrypt(text, columns);
    std::string decrypted = substitutionDecrypt(transposedBack);

    return decrypted;
}
```

```
===== CRIPTO-R1 =====
1. Caesar
2. Vigenere
3. Substituție
4. Transpoziție
5. Sistem mixt
0. Ieșire

Opțiune: 3

Textul pentru criptare:
CRIPTOGRAFIE

Textul criptat:
AQHOSNFQTDHC

Textul decriptat:
CRIPTOGRAFIE
```

4. Interfața și Validarea Datelor (Menu . cpp)

Pentru a elimina riscul crash-urilor sau al comportamentelor nedefinite (runtime errors), modulul care gestionează interfața interactivă a fost optimizat:

- **Tratarea excepțiilor de stream** (`cin.fail()`) : În cazul în care utilizatorul introduce caractere alfabetice în loc de numere la opțiunile meniului sau la cheia Caesar, starea de eroare a stream-ului este resetată (`cin.clear()`), iar bufferul rezidual este eliminat complet.
- **Validarea valorilor introduse:** Numărul de coloane utilizat la transpoziție și sistemul mixt este validat să fie $C \geq 1$, împiedicând apariția excepțiilor de tip împărțire la zero.

5.Concluzii

În cadrul acestui proiect au fost implementați și testați mai mulți algoritmi clasici de criptografie. Fiecare metodă permite atât criptarea, cât și decriptarea mesajelor introduse de utilizator. Implementarea modulară a permis separarea logicii fiecărui algoritm și organizarea eficientă a codului sursă. Prin realizarea acestui proiect au fost aprofundate conceptele fundamentale ale criptografiei clasice și modul de implementare al acestora în limbajul C++.

Link GitHub Repo

<https://github.com/madalina1teaca/university-cryptography-project/tree/main/CRIPTO-01>

UNIVERSITATEA TITU MAIORESCU

FACULTATEA DE INFORMATICĂ

Disciplina: Criptografie și Securitatea Informației

Activitate: Raport de Laborator — Referatul 1 (CRIPTO-R1)

Date Student

- **Nume și prenume:** Teacă Mădălina-Elena
- **Specializare:** Informatică
- **Coordonatori disciplină:** Prof. univ. dr. ing. Răcuciu Ciprian / Lector univ. dr. ing. Rogobete Marius